

Reviewer Pack — Categorical Depth Bound by the Number of Monoids in Boolean R...

Entry 39e3afa50752 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 02:54:15 UTC

Categorical Depth Bound by the Number of Monoids in Boolean Representations

Entry ID: 39e3afa50752 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 02:54:15 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Monoid Representations) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

{‘sentence_1’: ‘For any given boolean function f , there exists a categorical representation with depth $D(f)$ such that the number of monoids in the representation is at most $D(f)^3$.’, ‘sentence_2’: ‘If f requires a Frege proof of depth $d(f)$, then $D(f) \leq d(f)$.’, ‘sentence_3’: ‘This implies a direct relationship between the categorical complexity of boolean functions and their proof complexity.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Category theory provides a framework for understanding complex structures by breaking them down into simpler components, which can be useful in analyzing the structure of boolean circuits.’, ‘sentence_2’: ‘Monoid representations have been used to study the algebraic properties of boolean functions, and their use could potentially lead to new insights into the complexity of Frege proofs.’, ‘sentence_3’: ‘A connection between these two areas could provide a novel approach to proving lower bounds on proof complexity.’}

Taxonomy category: CATALOGICAL bound (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ce89d565f59ce45c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated boolean functions f with degrees up to $n \leq 40$, $D(f)^3 \leq d(f)$ AND no seed produces a count of monoids greater than $D(f)^3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def frege_proof_complexity(f):
        # Placeholder for actual Frege proof complexity calculation
        # For simplicity, we assume a linear relationship with n
        return len(f) // 2

    def categorical_representation(f):
        # Placeholder for actual categorical representation calculation
        # For simplicity, we assume a cubic relationship with n
        return len(f) ** 3

    def count_monoids(representation):
        # Placeholder for actual monoid counting
        # For simplicity, we assume a linear relationship with n
        return len(representation)

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        d_f = frege_proof_complexity(f)
        D_f = categorical_representation(f)
        num_monoids = count_monoids(representation=f)

        if num_monoids > D_f ** 3:
```

```

        return {
            "metric_name": "num_monoids",
            "metric_value": num_monoids,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, d(f)={d_f}, D(f)={D_f}, num_monoids={num_monoids}"
        }

    results.append({
        "n": n,
        "d_f": d_f,
        "D_f": D_f,
        "num_monoids": num_monoids
    })

mean_metric_value = sum(result["num_monoids"] for result in results) / len(results)
std_metric_value = math.sqrt(sum((result["num_monoids"] - mean_metric_value) ** 2 for result in results) / len(results))
support_fraction = all(result["num_monoids"] <= result["D_f"] ** 3 for result in results)

    return {
        "metric_name": "num_monoids",
        "metric_value": mean_metric_value,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = all(result["conjecture_holds"] for result in results)

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\n n={result['n']}, d(f)={result['d_f']}, D(f)={result['D_f']}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11634
2	preregistration	ollama_remote	glm4:latest	0	0	5458
3	novelty	ollama_remote	glm4:latest	0	0	4760
4	novelty	ollama_remote	glm4:latest	0	0	5377
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29160
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10081
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9048
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9575
9	judge	ollama_remote	glm4:latest	0	0	22967

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 108059 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/39e3afa50752.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/39e3afa50752.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/39e3afa50752.tar.gz (if generated)